

PROBLEM SOLVING SESSION

Ques Even subarrays

given integer array A.

↳ whether we can divide this array into one or more subarrays of even length such that first & last element of all subarrays should be even.

example

arr : 2, 4, 8, 6

sol.

$\begin{matrix} \downarrow & \downarrow & & \downarrow & \downarrow \\ [2, 4] & & [8, 6] \end{matrix}$
even even

YES

arr :

2, 4, 8, 7, 6

$\begin{matrix} [2, 4, 8] & [7, 6] \\ \text{odd} & \text{odd} \end{matrix} \quad \times$

$\begin{matrix} [2, 4] & [8, 7, 6] \\ \text{even} & \text{odd length} \end{matrix} \quad \times$

NO

arr

2, 4, 8, 7, 6

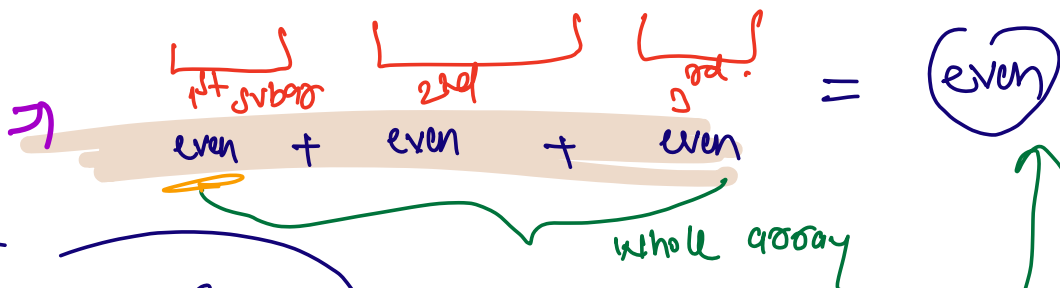
ans



a is always first element
g is last element.

- First element of array should be even
- End element " " should even

a, b, c, d, e, f, g



$$2 + 4 + 6 + 8 + 10 = \text{even}$$

length of original array should be EVEN

- First element even
- Last element even
- size of array even

Full array
as our
answer

YES

Code:

... / ...

string solve ("in")

{

if ($v[0] \% 2 == 0$ && $v[n-1] \% 2 == 0$)
 && $v[0] \% 2 == 0$)
 return "YES"

~~2, 4~~

2, 4, 6, 8, 9, 3

NO

else

return "NO"

sp & TC O(1)

Ques 2

Alternating Subarrays Easy

→ integer array A which contains 0's and 1's

→ integer B given

→ return an array which contains centers of all subarrays which is of length $2 * B + 1$

alternating subarrays.

→ subarray length



$2 * B + 1$ length.

0, 1, 0, 1, 0, 1 ✓

1, 0, 1, 0, 1, 1 ✓

1, 1, 0, 0, 1, 1 X

0, 1, 0, 1, 0, 0, X

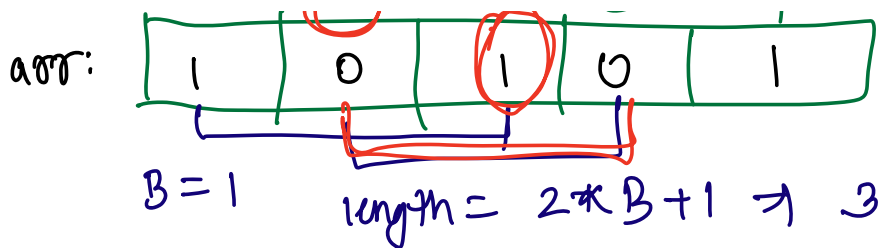
0

1

2

3

4



1st sub $\rightarrow$ 1 0 1 $\Rightarrow$ YES 1st index

2nd subarray = 0 1 0 $\Rightarrow$ YES 2nd index

3rd sub. $\rightarrow$

2	3	4
1	0	1

YES 3rd index

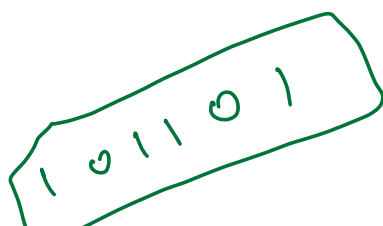
ans array $\rightarrow$ [1, 2, 3] as over ans array.

arr: 0 0 0 1 1 0 1

$B: 0$ $\text{length} = 2 * B + 1 = 1$

single element always alternative

ans array $\rightarrow$ [0, 1, 2, 3, 4, 5, 6]



$N \leq 10^3$ constraint

$0 \leq B \leq (N-1)$

1-

1/12

bool is-check-alternative (vector<int> A)

```
for (int i=0 ; i < A.size()-1 ; i++)
{
    if (A[i] == A[i+1])
        return false;
}
```

return True;

vector<int> ans;

⇒

```
for (int i=0 ; i < N-(2*B+1)+1 ; i++)
```

```
{
```

int start = i

int end = i + (2*B+1) - 1

left index

length

flag = 0

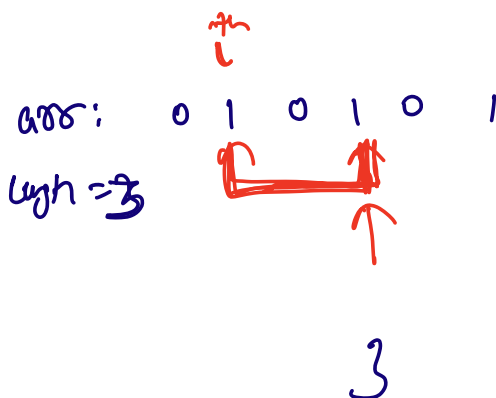
```
for (int j=start ; j < end-1 ; j++)
```

```
{
```

if (A[j] == A[j+1])
flag = 1

```
}
```

if (flag == 0) valid

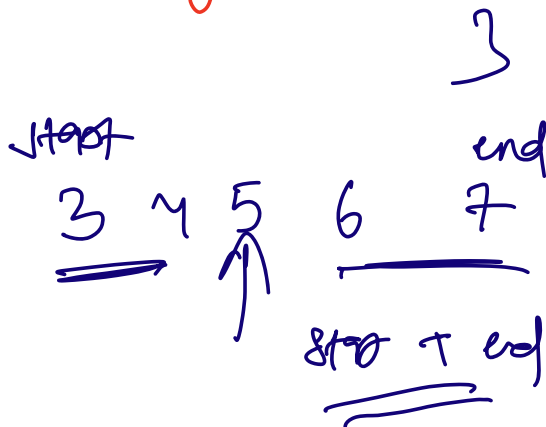


$$R - L + 1 = 2*B + 1$$

$$R - L + 1 = \text{length}$$

$$R = \text{length} + L - 1$$

ans. push-back (start+up)
2



$$\text{Time} = O(N \times N)$$

$$\text{space comp} = O(1)$$

BONUS Question

solve above question

$$N \leq 10^5$$

Time complexity $O(N)$

ques-3

LEADERS IN ARRAY

- integer array A of N distinct integers.
- Find leaders in array A.

leader: An element is a leader
 if it is strictly greater
 than all element to its
 right side. *rightmost
 always leader*

Example

arr:

16

,

17

,

4

,

3

,

5

,

2

ans:

[17, 5, 2]

Solution

BRUTE - FORCE APPROACH

for (int i = N-1; i >= 0; i--) {

int mx = 0

for (int j = i; j < N; j++)
{
 mx = max(mx, arr[j])
}

right
element

3

if (arr[i] == mx)

LEADER
ELEMENT
arr[i]

3

T.C = N^2

S.C = $O(1)$

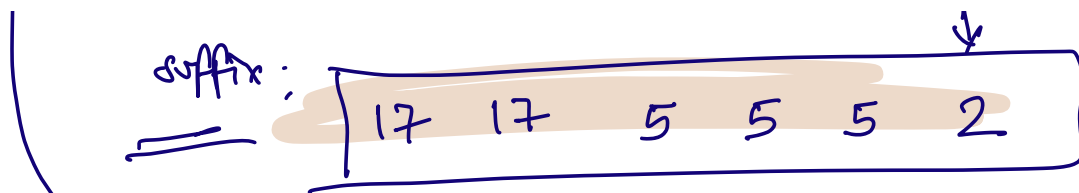
arr: 7, 8, 5, 3, 2, 5

suffi max: 8 8 5 5 5 5

Max. element from j to N

int suffix[N];

arr: 16 17 4 3 5 2



$O(N) \rightarrow O(1)$

on this array

T.C $\Rightarrow O(N) \Rightarrow$

S.C $\Rightarrow O(N) \Rightarrow$

Approach - 3

No storage of suffix array

just maintain one variable for max element

int mx = arr[n-1]

for (int i = N-2; i >= 0; i--)

{

mx = max(arr[i], mx)

if (mx == arr[i]) **LEADER**

}

space comp = $O(1)$

Time comp = $O(N)$

PICK FROM BOTH SIDES

Ques $\rightarrow$ Integer array A of size N

$\rightarrow$ Pick B elements

$\rightarrow$ left side or right side

(possibly 0 or more) (possibly 0 or more)

Max. obtained sum after pick B elements.

suppose $= B = 4$

$\rightarrow$ left = 4

right = 0

$\rightarrow$ left = 3

right = 1

left = 2

right = 2

left = 1

right = 3

left = 0

right = 4

arr:

5 -2 3 1 2

$B = 3$

$\rightarrow$
example
expln.

left = 3 $R = 0$ $\rightarrow 5 + (-2) + 3$
 $\rightarrow 6$

left = 2 $R = 1$ $\rightarrow 5 + (-2) +$
 (2)

left = 1 $R = 2$ $\rightarrow 5 + 1 + 2$

max

$\left\{ \begin{array}{l} \text{left} = 0 \quad R = 3 \rightarrow 3 + 1 + 2 \\ \rightarrow 6 \end{array} \right.$

ans. 8

Sol.

left = x element

right = B - x element

init Max_sum = 0

for (int x = 0; x <= B; x++)

init left = x

init right = B - x

init total_sum = 0

for (int i = 0; i < x; i++)

total_sum += arr[i]

left side

for (int i = B - x + 1; i < n; i++)

total_sum += arr[i]

Max_sum = max(Max_sum, total_sum)

return

3

optm max-sum

~~n~~ = 0

left = 0
right = 3

arr 5 -2 3 1 2
B = 3

left-side = 0

right-side = 6

total-sum = 6

max-sum = 6

n = 1

left = 1
right = 2

total = 5 + 1 + 2
= 8

Max-sum = 6

T.C = $O(N^2)$

S.C = $O(1)$

$O(N) \neq O(1)$

first loop = calculating sum

from 0 to N

second loop in calculating sum

from $B-N+1$ to N

$O(1)$

technique?

arr : 3 2 5 6 1 prefix
int prefix[]
sum : 3 5 10 16 17
prefix sum
 $O(1)$

arr : 3 2 5 6 1
int suffix[]
sum : 17 14 12 7 1
suffix sum
second loop
 $O(1)$

$T.C = O(N)$

S.C = $O(N)$

BONUS QUESTN

can we do above questi

T.C = $O(N)$

S.C $\rightarrow O(1)$

Hint $\rightarrow$ maintain variable

int prefix[]; int suffix[] $\leftarrow O(N)$
for (int x=0; x<=B; x++)

$\sum$

left = x

right = B-x

total sum of prefix[x-1]

$\rightarrow O(1)$

+ suffix[N-1-(B-x)+1]

$$\text{Max_sum} = \max(\text{total_sum}, \text{max_sum})$$

}

$$\begin{aligned} \text{T.C} &= O(1) \\ \text{S.C} &= O(n) \end{aligned}$$

Ques OBSERVATION BASED

- Given an integer array A, size N
- Integer B given.
- Return the array after rotating B times to the right.

arr = 1, 2, 3, 4

B = 2

res. array = 3, 4, 1, 2

arr = 2, 5, 6

B = 1

res. array =

⇒ ^{0th rotation} org. array = 1, 2, 3, 4

1st rotation $\rightarrow$ 4, 1, 2, 3
 2nd rotation $\rightarrow$ 3, 4, 1, 2
 3rd rotation $\rightarrow$ 2, 3, 4, 1
 4th rotation $\rightarrow$ 1, 2, 3, 4 $\Leftarrow$
 5th rotation $\rightarrow$ 4, 1, 2, 3

6th rotation
INTUITION $B = B \% N$ $O(1)$

100th rotation $\rightarrow$ $100 - 4$
 $\rightarrow 96 - 4$
 $B = 109$

arr:
 0 1 2 3
 1, 2, 3, 4
 2nd rotation $\rightarrow$ $0 \rightarrow 0+2 \rightarrow 2$
 2nd rotation $\rightarrow$ $B = 2$
 rev array.

i^{th} index $\rightarrow (i + B)$
 3rd index $\rightarrow 3 + 2$
 $\rightarrow 5$
 i^{th} index $\Rightarrow (i + B) \% N$

sol

```

vector<int> solve (vector<int> A)
{
    int ans[ ]
    int n = A.size();
    vector<int> ans(n)
    for (int i=0; i<n; i++)
    {
        ans[(i+B)%N] = A[i]
    }
    return ans;
}

```

$SC = O(N)$
 $TC = O(N)$

BONUS QUESTION

Can you chge above quest
space comp. $O(1)$

change your original array.

0th rot. a, b, c, d, e, f
 1st rot. f, a, b, c, d, e
 2nd e, b, a, b, c, d
 3rd d, e, b, a, b, c
 4th

$$B = 2$$

$$(N-B) = 4$$

$$B = 4$$

2

0, 1, 2, 3
1

2, 1

(Reverse)

B = 0, 1, 2, 3
4, 5, 6, 7
8, 9, 10, 11

N = 4

B = 0 % 4

2 0

B = 100th

100 % 4

2 2